

Documentation de Gestion de Projet — Vite & Gourmand

Contexte et objectifs

Le projet Vite & Gourmand a été réalisé dans le cadre d'une mission confiée par FastDev, entreprise de développement web, pour le traiteur bordelais Vite & Gourmand. L'objectif est de livrer une application web complète permettant d'afficher les menus et de gérer les commandes en ligne.

Durée du projet : Août — Septembre 2026

Développeur : Gaëtan Legrand

Client : Julie Martin et José Fernandez — Vite & Gourmand

Méthodologie

Le projet a été géré selon une approche **Agile** adaptée à un développeur solo. Les tâches ont été organisées en sprints hebdomadaires avec des livraisons progressives.

Outil de gestion de projet

Trello a été utilisé comme outil de gestion de projet. Le tableau est organisé avec des étiquettes de couleurs par catégorie :

- **Front-End** — composants React, maquettes, espaces utilisateurs
- **Base de données** — PostgreSQL, MongoDB, MCD
- **Back-End** — API REST, authentification, système de commandes
- **Déploiement / Sécurité** — mise en production, corrections sécurité
- **Transversal** — tests, documentation, Git

Le tableau Trello est accessible ici : <https://trello.com/b/vyotFWKA/vite-et-gourmand>

Phases du projet

Phase 1 — Analyse et conception (Semaine 1)

- Lecture et analyse de l'énoncé ECF
- Identification des besoins fonctionnels
- Conception du MCD (Modèle Conceptuel de Données)
- Création des maquettes sur Figma (3 desktop + 3 mobile)
- Export du code généré par Figma comme ossature du projet React
- Initialisation du repository GitHub

Phase 2 — Intégration Frontend (Semaine 2)

- Export et intégration de la maquette Figma
- Adaptation et personnalisation du code exporté
- Configuration de React + TypeScript + Vite
- Mise en place de TailwindCSS et ShadcnUI
- Développement des composants UI

Phase 3 — Backend et bases de données (Semaine 3)

- Création du schéma PostgreSQL
- Configuration de MongoDB Atlas
- Développement de l'API REST avec Express
- Mise en place de l'authentification JWT

Phase 4 — Connexion Frontend/Backend (Semaine 4)

- Création du service layer (authService, menuService, orderService)
- Connexion de l'authentification à l'API
- Connexion du système de commandes à PostgreSQL
- Configuration des emails avec Nodemailer

Phase 5 — Déploiement (Semaine 5)

- Déploiement du frontend sur Vercel
- Déploiement du backend sur Render
- Configuration PostgreSQL sur Render

- Configuration MongoDB sur Atlas
- Tests en production

Phase 6 — Documentation (Semaine 6)

- Documentation technique
 - Manuel utilisateur
 - Charte graphique
 - Diagrammes UML
 - Gestion de projet PDF
 - Journal de bord PDF
 - Refactorisation App.tsx (1500 → 308 lignes)
 - Corrections de sécurité
-

Difficultés rencontrées et solutions

Problème d'encodage Windows/PostgreSQL

Les caractères accentués dans les ENUMs PostgreSQL étaient mal encodés sur Windows.

Solution : utilisation de chcp 65001 avant psql et renommage des valeurs via ALTER TYPE ... RENAME VALUE.

Gestion des variables d'environnement

Le package dotenvx interférait avec le chargement standard des variables. Solution : ajout de valeurs par défaut dans le code avec l'opérateur ||.

CORS en production

Le frontend Vercel ne pouvait pas appeler le backend Render à cause de la politique CORS.

Solution : configuration explicite des origines autorisées dans Express.

Emails en production

Render bloque les connexions SMTP sortantes même sur le plan payant. Solution : les emails fonctionnent en local, un service SMTP externe (Brevo ou SendGrid) serait nécessaire en production.

Fichier App.tsx volumineux

Le fichier principal du frontend contenait plus de 1500 lignes, rendant les modifications risquées. Solution : refactorisation complète — App.tsx découpé en 17 composants séparés, réduisant le fichier à 308 lignes.

Sécurité — credentials en dur

Des mots de passe et secrets étaient présents en dur dans le code. Solution : suppression de toutes les valeurs sensibles, utilisation exclusive des variables d'environnement et correction de la faille IDOR.

Encodage des statuts PostgreSQL

Les ENUMs de statuts de commande étaient corrompus en BDD (accents Windows). Solution : normalisation Unicode dans le code backend pour comparer les statuts sans tenir compte des accents.

Livrables

Livrable	Statut	Lien
Repository GitHub public	✓	https://github.com/gaetanlegranddev-svg/vite-et-gourmand
Application déployée	✓	https://vite-et-gourmand-red.vercel.app
Tableau Trello	✓	https://trello.com/b/vyotFWKA/vite-et-gourmand
Fichier SQL	✓	database.sql dans le repo
Manuel utilisateur PDF	✓	Manuel_Utilisateur_Vite_Gourmand.pdf
Charte graphique PDF	✓	Charte_Graphique_Vite_Gourmand.pdf
Documentation technique PDF	✓	Documentation_Technique_Vite_Gourmand.pdf